

Atención en RNNs

Fundamentos del Procesamiento del Lenguaje Natural

Grado en Ciencia de Datos e Inteligencia Artificial

Universidad de Deusto

Curso 2025-26

Motivación

- ▶ **Bottleneck del Encoder:** En modelos *seq2seq* tradicionales, todo el input se comprime en el último *hidden-state* h_T del encoder $\Rightarrow$ dificultad para captar dependencias a largo plazo.
- ▶ **Memoria selectiva:** Al generar y_t , no toda la entrada es igual de relevante; queremos *enfocarnos* en las partes más útiles y relevantes de la secuencia de entrada.
- ▶ **Beneficio práctico:** Mejor calidad de traducción y generación de texto, especialmente en secuencias largas.

Idea: En cada paso t del decoder, se construye un **vector de contexto** c_t como mezcla ponderada de *todos* los estados del encoder $\{h_1, \dots, h_T\}$.

Cálculo de Atención (Paso a Paso)

Sea s_{t-1} el estado del decoder en $(t - 1)$ y h_i los estados del encoder:

1. Puntuaciones (Attention Scores):

$$e_{t,i} = \text{score}(s_{t-1}, h_i) \quad \begin{cases} \text{Dot (Luong): } e_{t,i} = s_{t-1}^\top h_i \\ \text{General: } e_{t,i} = s_{t-1}^\top W_a h_i \\ \text{Additive: } e_{t,i} = v_a^\top \tanh(W_s s_{t-1} + W_h h_i) \end{cases}$$

2. Pesos de Atención (Attention Weights):

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{k=1}^T \exp(e_{t,k})}, \quad \sum_i \alpha_{t,i} = 1$$

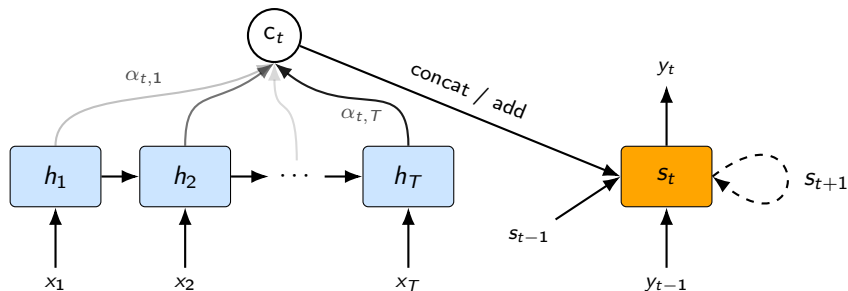
3. Vector de Contexto (Context Vector c_t):

$$c_t = \sum_{i=1}^T \alpha_{t,i} h_i$$

4. Estado del Decoder y Distribución de Salida:

$$s_t = f_{\text{RNN}}(s_{t-1}, y_{t-1}, c_t), \quad p(y_t \mid y_{<t}, x) = \text{softmax}(W_o [s_t; c_t])$$

Encoder-Decoder con Atención (Ilustración)



$$\alpha_{t,i} = \text{softmax}(\text{score}(s_{t-1}, h_i))$$
$$c_t = \sum_i \alpha_{t,i} h_i$$

La opacidad de las flechas $h_i \rightarrow c_t$ representa la magnitud de los pesos de atención $\alpha_{t,i}$.

Intuición de los Pesos de Atención $\alpha_{t,i}$

Idea: Cuando el *decoder* genera la palabra $y_t = \text{“casa”}$, no todas las posiciones del *encoder* son igual de importantes.

Cada peso $\alpha_{t,i}$ indica cuánto “mira” el modelo la palabra fuente x_i en ese paso de generación.

Ejemplo:

- ▶ Frase origen (inglés): *I went home yesterday.*
- ▶ Frase destino (español): *Fui a casa ayer.*

En el paso en que se genera “casa”, el modelo debería “fijarse” en “home”:

$\alpha_t = (0,05, 0,08, 0,70, 0,17) \Rightarrow c_t = 0,70 h_{\text{home}} + \text{otros términos pequeños.}$

La atención elimina el cuello de botella del último estado del encoder, permitiendo al modelo “mirar” directamente las partes más relevantes de la entrada.

Variantes de *Scoring Functions*

Además de las funciones **Dot**, **General** y **Additive** vistas anteriormente, existen otras variantes para calcular los *scores* de atención $e_{t,i}$:

Función	Fórmula de $e_{t,i}$	Parámetros
Dot (Luong)	$e_{t,i} = \mathbf{s}_{t-1}^\top \mathbf{h}_i$	Ninguno
General	$e_{t,i} = \mathbf{s}_{t-1}^\top \mathbf{W}_a \mathbf{h}_i$	$\mathbf{W}_a \in \mathbb{R}^{d_s \times d_h}$
Additive (Bahdanau)	$e_{t,i} = \mathbf{v}_a^\top \tanh(\mathbf{W}_s \mathbf{s}_{t-1} + \mathbf{W}_h \mathbf{h}_i)$	$\mathbf{W}_s \in \mathbb{R}^{d_a \times d_s}$, $\mathbf{W}_h \in \mathbb{R}^{d_a \times d_h}$, $\mathbf{v}_a \in \mathbb{R}^{d_a}$
Scaled Dot-Product	$e_{t,i} = \frac{\mathbf{s}_{t-1}^\top \mathbf{h}_i}{\sqrt{d_k}}$	Escala por $\sqrt{d_k}$ el Dot de Luong
Bilinear	$e_{t,i} = \mathbf{h}_i^\top \mathbf{W}_b \mathbf{s}_{t-1}$	$\mathbf{W}_b \in \mathbb{R}^{d_h \times d_s}$
Cosine Similarity	$e_{t,i} = \frac{\mathbf{s}_{t-1}^\top \mathbf{h}_i}{\ \mathbf{s}_{t-1}\ \ \mathbf{h}_i\ }$	Normalización de vectores
Multi-Head Attention	$e_{t,i}^{(h)} = \frac{(\mathbf{W}_q^{(h)} \mathbf{s}_{t-1})^\top (\mathbf{W}_k^{(h)} \mathbf{h}_i)}{\sqrt{d_k}}$	$\mathbf{W}_q^{(h)}, \mathbf{W}_k^{(h)} \in \mathbb{R}^{d_k \times d_s}$, múltiples cabezas h

Tenéis una explicación más detallada de cada una de estas funciones [aquí](#).

Ejemplo de Implementación en PyTorch

Vamos a ver a continuación un ejemplo sencillo de cómo implementar un mecanismo de atención (e.g., Bahdanau) en **PyTorch**.

```
import torch
import torch.nn as nn

class BahdanauAttention(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.Wa = nn.Linear(hidden_size, hidden_size)
        self.Ua = nn.Linear(hidden_size, hidden_size)
        self.Va = nn.Linear(hidden_size, 1)

    def forward(self, query, keys):
        scores = self.Va(torch.tanh(self.Wa(query) + self.Ua(keys)))
        scores = scores.squeeze(2).unsqueeze(1)
        attention_weights = torch.softmax(scores, dim=-1)
        context = torch.bmm(attention_weights, keys)
        return context, attention_weights
```

Si queréis consultar una implementación completa, podéis ver este [post](#).

RNNs con Atención $\Rightarrow$ ¿Por qué Transformers?

Lo que corrige la atención: el cuello de botella del último estado del encoder y la selección dinámica de información relevante.

... pero siguen existiendo limitaciones importantes:

- ▶ **Secuencialidad dura:** el encoder/decoder de RNNs procesa paso a paso $\Rightarrow$ *poca paralelización* en entrenamiento e inferencia.
- ▶ **Rutas de gradiente largas:** incluso con LSTMs/GRUs, las dependencias muy largas siguen siendo complejas...
- ▶ **Coste temporal:** con atención global, cada paso del decoder compara contra *todos* los h_i del encoder.
- ▶ **Aprovechamiento de hardware:** la recurrencia limita el beneficio de GPUs/TPUs frente a operaciones masivamente paralelas...

Next Steps: Reemplazar la recurrencia por **auto-atención paralelizable**: cualquier posición se conecta en *una* capa + eliminar la autoregresión.

¿Preguntas?